
Modular Programming – Practical Work

Week 4

Classes and Objects, a deeper look

Notations: W1Q1 = Week 1, Questions of type 1

Question types:

- Type 1: theory, concepts *knowledge*
- Type 2: understanding typically based on code reading *understanding*
- Type 3: producing yourself code *know-how*

A moodle deposit space is open every week for you to upload your solutions. Solutions of the exercises are provided one week later.

Important points:

- 602_F and 602_D classes should upload solutions **before Tuesday 24h00 the following week**
- **2 or 3 practical work will be corrected** and will count for one third of the intermediate test note

Conventions about fonts:

- Lucida Console is used for Java reserved commands
- Courier is used for the code
- Arial is used for the text

W4Q1 – 1

1. Why is the main method of a Java application declared as `static` ?

At the beginning of the execution of a Java program, the Java virtual machine looks for a static method called `main`. The main method is declared static because at the beginning of the execution, there are no object instances and therefore, the entry point has to be a static method.

2. Can we refer to `this` in a static method?

No, we can't refer to `this` in a static method. Referring to `this` would mean that we should always have an object instance of the class, which is, by definition of a static method, not the case.

3. `static`: In a static method, can we access to

- an instance variable → no
- a class variable → yes
- a local variable of another method → no
- a local variable of this static method → yes

4. Weshalb ist es allgemein gesehen vorzuziehen, eine Instanzvariable mit einem `private` Zugriff zu deklarieren?

Es ist immer besser, mit dem Verkapselungsprinzip zu arbeiten, d. h. die Details der Implementierung zu verbergen (*Clients* der Klasse).

Wenn auf die Instanzvariable trotzdem von aussen her zugegriffen werden muss, wird die Verwendung von *Gettern* und *Settern* empfohlen, um den Wert zu erhalten bzw. ihn zu verändern. Bei der Veränderung kann mithilfe des *Setters* überprüft werden, ob der Wert, der in der Instanzvariable gespeichert wird, zugelassen ist (bei einer Instanzvariable des Typs `int`, die eine Zeit darstellt, sind negative Werte oder Werte grösser als 24 zum Beispiel nicht erlaubt).

5. Wozu dient das Überladen (Overloading) von Konstruktoren?

Das Überladen von Konstruktoren ermöglicht die Definition von mehreren Konstruktoren, welche einen verschiedenen Satz von Eingangsparametern übernehmen.

Beispiel: In der Klasse `watchPin` aus der Übung W3Q2 können zwei Konstruktoren definiert werden: ein Konstruktor wie aus der Vorlesung und ein anderer mit denselben Parametern sowie einem weiteren Konstruktor, der den *Offset* `o` definiert. So können dieselben Konstruktionsfunktionen verwendet werden (indem der erste Konstruktor beibehalten wird), ohne dass der Code der anderen Klassen, die `watchPin` aufrufen, geändert werden muss. Gleichzeitig können die Kapazitäten von `watchPin` über einen anderen Konstruktor erweitert werden.

W4Q1 – 2

In the following table, find the correspondences between the terms of column 1 and the descriptions of column 2.

Term		Description
g	final	a) Called to initialize the objects of a given class.
h	Array	b) The fact of limiting the access to members of a class using <code>private</code> and to control the access to these members using <code>public</code> methods.
f	Attributes	c) Method called by the <i>Garbage Collector</i> when the memory is cleaned from the objects without references.
j	Class	d) Is used to make a reference to instance variables and methods of the current object.
a	Constructor	e) Modifier used to make a variable unique for all objects of a given class.
i	Object	f) Another name for instance variables.
b	Encapsulation	g) Modifier to make a variable un-modifiable (constant).
c	finalize	h) Group of variables containing objects of the same type.
d	this	i) Instance of a class.
e	static variable	j) Define the common attributes and behaviors of objects.

W4Q3 – 1 Héritage

Folgende Fahrzeuggruppe soll modelliert werden:

Plane, Boat, Carriage, Truck, Motorcycle, Sled, Bicycle, Sailboat, Car

(Auto, Boot, Fahrrad, Flugzeug, Kutsche, LKW, Motorrad, Schlitten, Segelboot)

Jedes Fahrzeug besteht aus verschiedenen Komponenten:

Reactor, Wheel, Engine, Handlebars, Oar, Rudder, IceSkating, Horse, Sail, SteeringWheel, Trailer

(Triebwerk, Rad, Motor, Lenkstange, Ruder, Steuerruder, Kufe, Pferd, Segel, Steuerrad, Anhänger)

Diese Komponenten sind auf dem Netzwerkbereich verfügbar: Moodle (E-Learning) !

Sie sollen diese jedoch nicht verändern, sondern nur benutzen!!!

Das Ziel dieser Übung besteht darin, die Fahrzeugklassen sowie, falls nötig, deren jeweiligen Oberklassen zu schreiben und dabei die Redundanz möglichst zu **reduzieren**. Erstellen Sie überall dort, wo es möglich ist, eine Oberklasse, welche die Teile enthält, die auch andere Klassen **gemeinsam** haben.

Schreiben Sie die Klassen „Fahrzeuge“ und **verwenden Sie die am besten geeignete Vererbung**, um das nachstehende Ergebnis zu erzielen.

Wenn Sie die Datei Exercice_14.java ausführen, sollen Sie folgendes Ergebnis erhalten:

```
-----
Plane reactor :
Reactor power : 3500.0
Wheel of : 28.0 inch(es)
Wheel of : 28.0 inch(es)
-----
Boat of 2 oars :
Oar width : 0.25
Oar width : 0.25
Rudder depth : 0.85
-----
Carriage of 4 horses :
Horse race : Camargue
Horse race : Camargue
Horse race : Camargue
Horse race : Camargue
Wheel of : 77.0 inch(es)
Wheel of : 77.0 inch(es)
Wheel of : 77.0 inch(es)
Wheel of : 77.0 inch(es)
-----
Truck trailer :
Trailer weight : 27.0
Steering wheel diameter : 0.38
Engine of : 4.85 liter
Wheel of : 35.0 inch(es)
Wheel of : 35.0 inch(es)
Wheel of : 35.0 inch(es)
Wheel of : 35.0 inch(es)
-----
Motorcycle :
Engine of : 0.75 liter
Handlebars width : 0.45
Wheel of : 6.0 inch(es)
Wheel of : 6.0 inch(es)
-----
```

Sled with two iceSkatings:

IceSkating length : 3.5

IceSkating length : 3.5

Horse race : Percheron

Horse race : Percheron

Horse race : Percheron

Horse race : Percheron

Bicycle :

Handlebars width : 0.25

Wheel of : 1.5 inch(es)

Wheel of : 1.5 inch(es)

Sailboat :

Steering wheel diameter : 0.0

Rudder depth : 2.15

Car :

Steering wheel diameter : 0.32

Engine of : 1.65 liter

Wheel of : 18.0 inch(es)

Wheel of : 18.0 inch(es)

Wheel of : 18.0 inch(es)

Wheel of : 18.0 inch(es)

W4Q3 – 2

In the next exercise session you will have to complete a battle system for a very primitive text-mode role playing game. In this exercise, you will design several classes of characters and expose the hierarchy (text or Visio class diagram).

The base class for all the battlers is the class `Character`. Can you design a hierarchy of classes where the top class is `Character` and where there is a distinction between good and bad guys? Design your solution taking into account the good and evil.

Here are the classes that you must have in your hierarchy:

GOOD:

Player (Human).

EVIL:

Ogre (Non Human), Troll (Non Human), Barbarian (Human), White Mage (Human) and Black Mage (Human).

Player is a good character, while all the others are enemies.

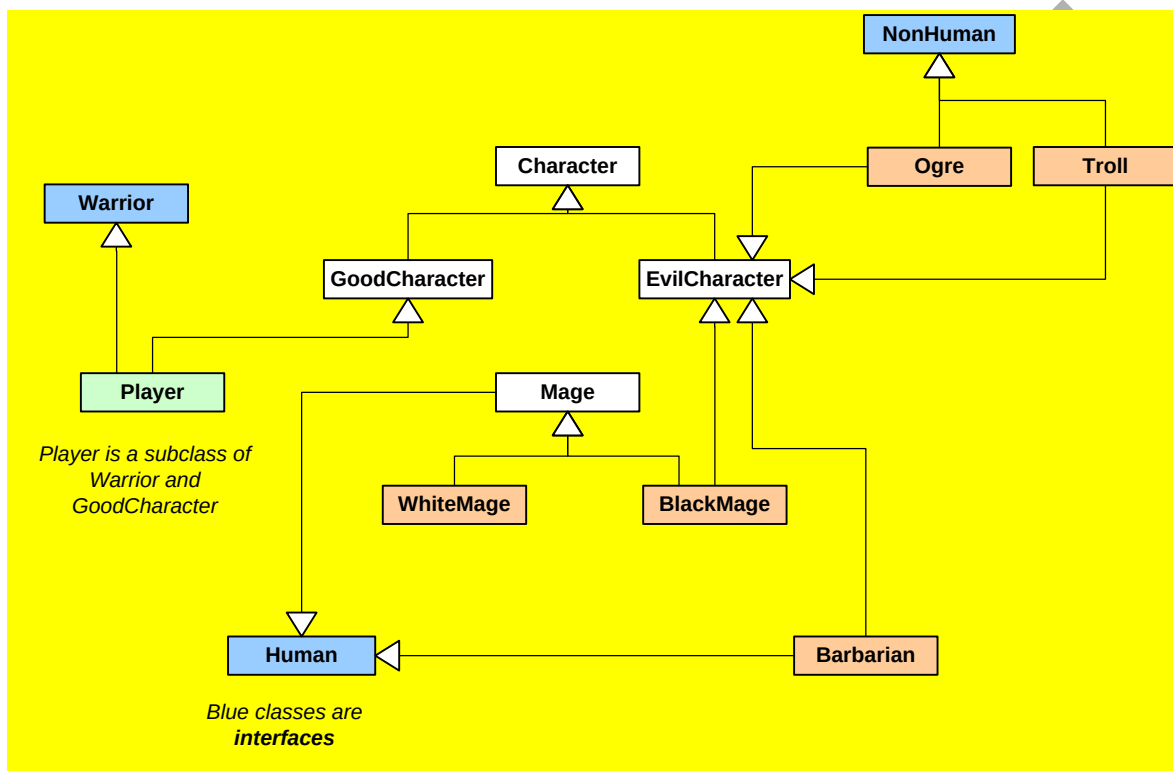
You can also consider that `WhiteMage` and `BlackMage` are subclasses of `Mage`. It's up to you to design the solution you find the best.

Remarks:

The goal of this exercise is to show you that there is no best solution when working with inheritance. This is the reason why all projects using Object-oriented approaches must

first be designed. This is to ensure that the solution you've found correctly covers your needs.

Solution



This solution emphasizes on the use of interfaces. These are represented by blue boxes. The use of interfaces will be given further in the course. The goal of the exercise was to search for yourself on the Internet for the pieces of information you need.

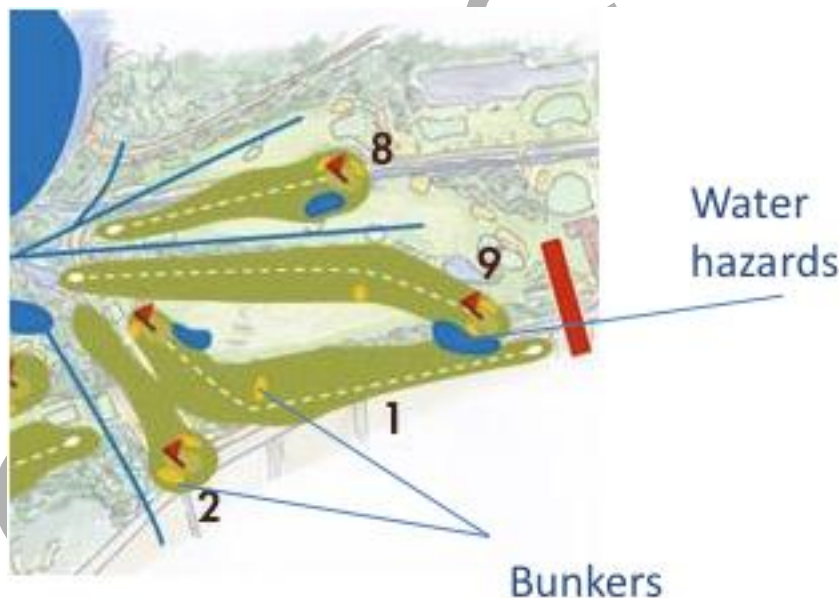
Good searching skills are an absolute must have for programmers!

W4Q3 – 3

The company employing you receives as mandate to redo the website of a golf club: <http://www.golfsierre.ch/fr/>. Your boss believes that similar type of golf club clients will mandate the company to build their websites. Therefore, he opts for a clean implementation that could be re-used for other clients. The boss asks you to propose a basis class design that represents what a golf is and he asks you to do the implementation in Java technology¹.

As you can see during your visit, a golf ("parcours de golf") has a given number of holes, typically 9 or 18. Looking at hole descriptions in the web site, you identify different elements that define the entity "hole" (<http://www.golfsierre.ch/fr/parcours/golf1.php>) : hole description, handicap, men distance, ladies distance, par (number of hits you are supposed to accomplish), etc.

A golf player explains you that a hole is also identified by hazards such as bunkers (sand traps) or water hazards. A water hazard is typically shared between different holes, as illustrated below.



1. Propose a class design.
Your objective in doing so is to be able to recompose all the information available on the hole description web pages of the golf of Sierre.
2. Implement the classes in Java.
3. In each class, provide a toString() method that returns a String representing the content of the class.

¹ Someone else in the company will be in charge of the html page generation on top of your class (for ex using JSP or simple Java servlets).